

ECS 152A: Computer Networks

Winter 2026

Project 1

(35 points)

Logistics

Due Date: Friday February 13th, 5:00pm PT

Team: The project is to be done in a team of 2 students. You *cannot* discuss your code/data with other classmates (*except* your project partner).

All submissions (including your code) will be checked for **plagiarism** against other submissions as well as the public Internet. *This includes submissions from previous quarters.* Plagiarized submissions will be entitled to **zero** points and may result in the students involved failing the class.

Don't cheat. It's not worth it.

Both the bare bones description of the project, and the lack of starter code, are intentional. The initial pain in getting started and connecting the project with the course material is certainly a bit jarring, but it will force you to define the implementation requirements, which is a central skill in computer networking. Even today, when the core protocols of the Internet have been around for decades, this skill is essential for application developers who need to use the network (and what application does not?).

TAs and the instructor are more than happy to work with you on your own implementation and give you tips. We have 5 office hours every week, and both the TAs and the instructor would be more than happy to schedule additional meetings by appointment if needed. The TAs are the best resource when it comes to the project implementation, and the instructor can help you connect the project to the course content.

Note for group submissions: The three parts of this assignment are independent of each other. If you're doing this assignment as a team - you can divide the work by assigning a part to an individual or split a part. Please write your individual contributions on the last page before signing.

Congestion Control

This question was adapted from Zubair Shafiq's class. Your goal is to implement the **UDP sender** in Python that will send data from the large file to the receiver in a series of packets, where the size of each packet is 1024 bytes.

In this project, you are given a docker container that contains a large file (file.mp3), a UDP receiver (that you will start and then send data to), and an emulated network profile (called the training profile). The network profile will set buffer size, introduce delays, and determine the throughput of the link between the sender and the receiver. You can find the docker container with the file, receiver, and network profile at [this link](#). The link contains a README with instructions on scripts to build and deploy the container.

You will implement 3 variants of the sender that implement the following congestion control protocols:

1. **Stop-and-wait protocol (15 points)**
2. **Fixed sliding window protocol with size 100 packets (20 points)**
3. **TCP Reno (Extra Credit)**

For each UDP sender, you will measure and report the (a) throughput (size of transmitted data/time taken to send data) in the units of bytes per second , (b) the average per-packet delay in the units of seconds and at the end, (c) the performance of the UDP sender.

- (a) To measure **throughput**, start your timer as soon as you create your socket and stop your timer once you have received acknowledgments for all packets. You have to include sequence numbers in your packets to keep track of acknowledgments.
- (b) To measure the **per-packet delay**, you will start your timer when you send the packet and stop the timer when you receive an acknowledgment from the receiver for that packet. In case of retransmissions, you should consider the timer to start when you send the packet the first time and stop the timer when you finally receive the acknowledgement.
- (c) To evaluate the **performance** of your UDP sender, you are required to compute the following metric:

$$\text{Metric} = 0.3 \text{ Throughput}/1000 + 0.7/\text{Average Delay per Packet}$$

For TCP Reno, set the initial window size = 1 packet and the initial slow start threshold to 64 packets.

Due to inherent randomness in the network profile, you are required to measure the throughput, average delay, and performance metric 10 times. Each program should only output 3 lines: the average throughput (in bytes per second), the average packet delay (in seconds), and the average performance metric separated by a comma. All numbers should be reported as floating points, rounded up to 7 decimal points with no units.

Note:

- In your final submission, make sure you remove all print/debugging statements since it could impact the performance of your implementation as measured by our wrapper testing program.
- All submissions will be tested on Python 3+.
- You can use ChatGPT or another LLM for guidance on implementing your protocols and writing your report, but you have to disclose it following [ACM rules](#).
- If an implementation or report appear entirely copied from ChatGPT or a similar tool, we may call you in for a meeting and you will need to explain your code to us.

In the report,

1. You have to describe your implementation in at least 200 words. Without an adequate description, your submission will be considered invalid. There is no upper word limit for the description of protocols.
2. You should also include a table of throughput, per-packet delay, and the performance metric for each protocol on the training profile. This table would contain the average and standard deviation of 10 measurements.
3. Add each of your output screenshots in the report.

Your submission should contain:

1. Code:
 - a. **sender_stop_and_wait_[name1]_[student_id1]_[name2]_[student_id2].py**
 - b. **sender_fixed_sliding_window_[name1]_[student_id1]_[name2]_[student_id2].py**
 - c. **sender_reno_[name1]_[student_id1]_[name2]_[student_id2].py** (optional extra credit)
2. Report:
 - a. **proj1_sender_[name1]_[student_id1]_[name2]_[student_id2].pdf**

Rubric

Code (25 Points)

1. Stop and Wait Protocol (9 Points):

- a. Correct number of packets sent (3)
- b. Sequence of packets managed correctly – Is the packet reconstructed correctly in case of errors? (3)
- c. Metrics measured correctly over 10 iterations (3)

2. Fixed Sliding Window Protocol (16 Points):

- a. Correct number of packets sent in each cycle (3)
- b. The window pointers update correctly (4)
- c. Packet loss handled correctly (4)
- d. Metrics measured correctly over 10 iterations (3)

Report (10 Points)

1. Stop and Wait Protocol (4 Points):

- a. Technical explanation, including citing external sources, if any; mention "none" otherwise (3)
- b. Results (1)

2. Fixed Sliding Window Protocol (6 Points):

- a. Technical explanation (4)
- b. Window adjustment technique explained (2)
- c. Citing external sources, if any; mention "none" otherwise (1)
- d. Results (1)

Extra Credit - TCP Reno Section

1. Technical Implementation and Explanation (70% of extra credit)
 - a. How was the slow-start threshold established?
 - b. Implementation of AIMD (Additive Increase Multiplicative Decrease) – How was the congestion window size increased?
 - c. How were fast retransmit and fast recovery implemented?
2. Handling Congestion with Examples (15%)
3. Citing External Sources (5%)
4. Metrics Computation Over 10 Iterations (10%)

How extra credit will be given:

- **[70-100%] Correct:** Increase the lowest (non-dropped) quiz grade to 100% *or* improve the final course grade by a full letter modifier (whichever is greater).
- **[50-70%] Correct:** 2 points extra credit in the class.
- **[25-50%] Correct:** 1 point extra credit in the class.

Late Submission Policy:

No late submissions are allowed. However, if you barely miss the deadline, you can get partial points up to 24 hours. The percentage of points you will lose is given by the equation below. This will give you partial points up to 24 hours after the due date and penalize you less if you narrowly miss the deadline.

$$\textit{Total Marks you get} = (\textit{Actual Marks you would get if NOT late}) * [1 - \textit{hours late}/24]$$

Late Submissions (later than 24 hours from the due date) will result in zero points *unless you have our prior permission or documented accommodation.*

Best of luck

Submission Page

Include this signed page with your submission

I certify that all submitted work is my own work. I have completed all of the assignments on my own without assistance from others except as indicated by appropriate citation. I have read and understand the [university policy on plagiarism and academic dishonesty](#). I further understand that official sanctions will be imposed if there is any evidence of academic dishonesty in this work. I certify that the above statements are true.

For Group Submissions,

Team Member 1's contributions to this assignment:

Short Summary:

Full Name	Signature	Date
-----------	-----------	------

Team Member 2's contributions to this assignment:

Short Summary:

Full Name	Signature	Date
-----------	-----------	------